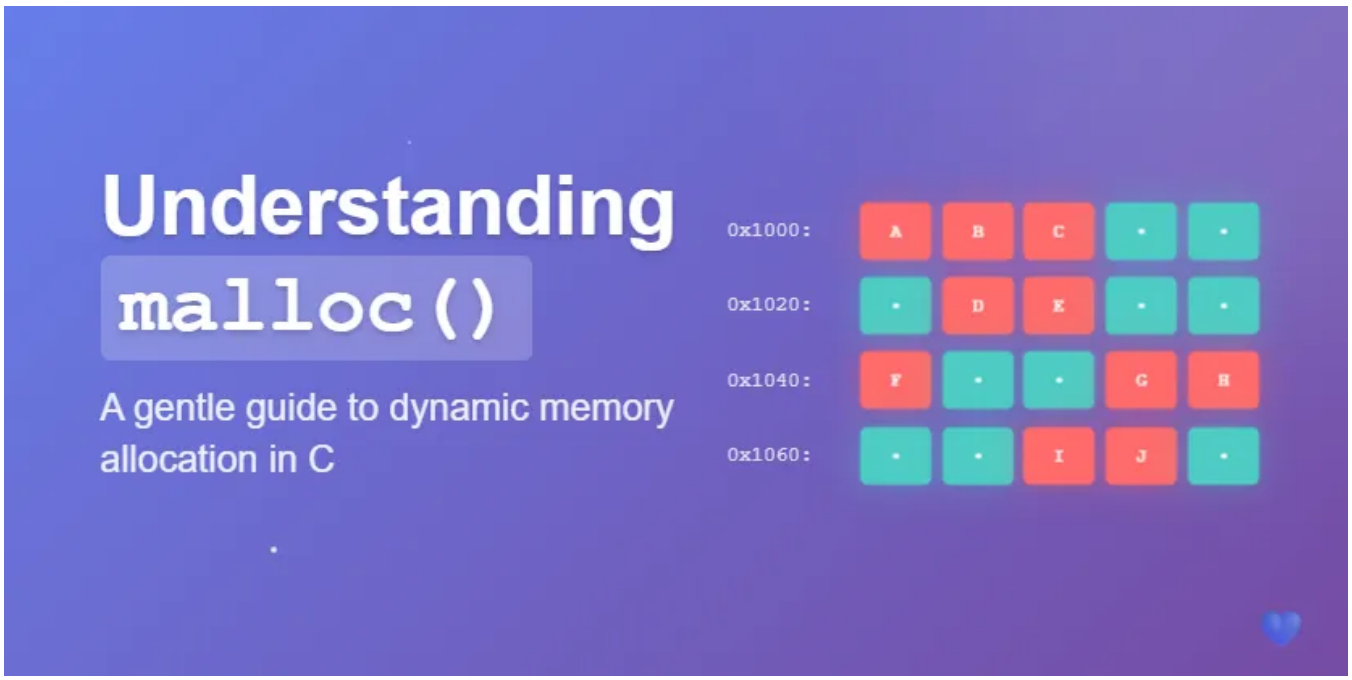
How does a malloc work?

13 min read · Jul 24, 2025

Aniket Kumar Follow

Listen Share



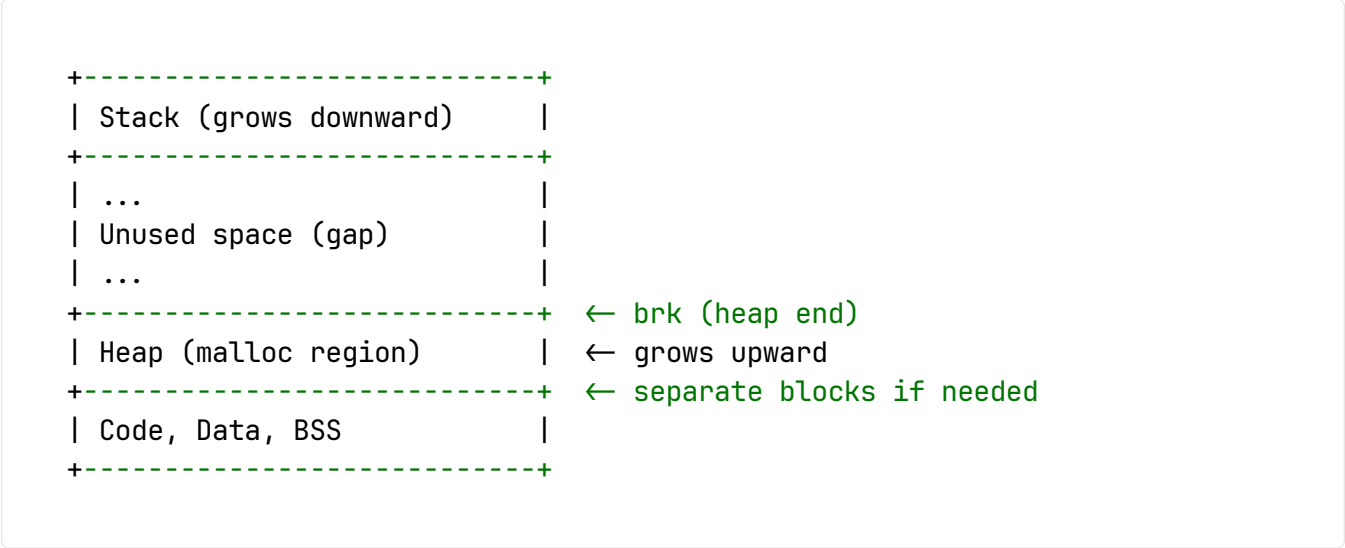
Malloc is a standard library function that allows for dynamic memory allocation. It provides a mechanism for programs to request blocks of memory from the heap at runtime, rather than relying solely on the fixed-size allocations determined at compile time (like global or stack variables).

The Heap: `malloc()`'s Playground

To understand `malloc()`, we must first understand the "**heap.**"

The heap is a region of memory set aside by the operating system for dynamic memory allocation.

Let’s first start by understanding the memory layout.



brk, is just a pointer that define the **heap boundary**. Think of this like a boundary till what point the heap is allocated for the given process.

The heap’s physical memory is not **pre-allocated.** Pages are committed lazily when first touched (demand-paging).

When you call `malloc()`, you're essentially asking the operating system (indirectly) for a chunk from this heap memory.

The **heap** is part of the **process’s virtual address space**.

- The **OS only gives memory to a process on request**
- The **standard library** (like `glibc`) manages the heap **on top of OS memory.**

What Happens When You Call `malloc(size)`?

While you simply pass a `size` argument to `malloc()`, a series of intricate steps are set in motion:

We might have two cases over here.

(i) The memory might be available within our heap and handler might return the memory from here

(ii) The heap does not have sufficient memory for the requested size.

Let's discuss the first case:

When you call `malloc()` the request is handled entirely in user space by **glibc allocator** (**ptmalloc**) before the kernel ever sees it. ptmalloc is the specific allocator used by the **glibc**.

Glibc **tries multiple sources** to satisfy the request **in this exact order** (for small-to-medium sizes):

```
tcache → fast bin → unsorted bin → small/large bins → top chunk → syscall
```

The initial heap for a process is managed by the '**Main Arena**,' which is created at program startup. This is the central arena used for dynamic memory management in a process. The main arena manages memory via **sbrk(), but if the allocation request is really large, it may use *mmap* to serve such large request.**

Now, try to understand, mostly we all must have used malloc in a single thread application.

But when we are working in multi-threaded environment.

Multiple thread might be issuing malloc call, if all the request directly reach the OS, then its problematics, because there might be lock contention. Main arena becomes a bottleneck here, which leads to the need for thread-local

arenas.

T**hread arenas** as well or sometime called **secondary arena.** Thread arenas sit exactly between those two ideas: they are still **user-space structures**, but they exist because multi-threaded programs would otherwise fight over the single main arena.

Unike, Unlike the main arena, thread arenas obtain their memory via **`mmap()`**-ed regions (anonymous mappings), not the traditional heap.

Glib has several internal structure internally to manage **dynamic memory allocation** efficiently. These '**bins**' are internal data structures (**linked lists**) that hold **free memory chunks,** making it faster to handle subsequent **allocations** and **deallocations** and reducing **fragmentation**.

The data for the main arena is stored in a global variable in the **libc data segment** (not part of the heap segment itself). It maintains multiple fields including **mutexes**, **fastbins**, **normal bins**, the **top chunk pointer**, and **bookkeeping counters**. Its **`malloc_state`** struct holds all the needed state to manage the main heap's allocations and free lists, which we shall discuss below.

1. Per-thread (tcache)

Each thread in your program gets its own private memory cache (`tcache`) to serve small `malloc` / `free` operations **without locking**.

`tcache` breaks allocations into **size classes**, where each class represents a different chunk size. Think of this as bucket of different size of memory.

For example:

- Size class 1 → 16 bytes
- Size class 2 → 32 bytes
- Size class 3 → 48 bytes
- This typically goes up to size classes for chunks around 1 KiB (e.g., up to 1024 bytes).

When program encounters `malloc()`, glibc will first check the **tcache** bin. if there's a free chunk available in the correct size class.

This is a significant optimization as it avoids costly system calls and mutex locks, greatly improving performance for frequent small allocations.

Now, lets say tcache is not able to provide the given memory, than the request goes to below **bins**.

2. Fast Bins

Size range: Typically upto 64 bytes on a 64 bit system.

(≤ 64 B): Designed for speed, this is usually for a very smaller allocation. If the request size is less than 64 byte, than the request is served from here. Each **fastbin** is basically a singly linked list of freed chunks of the same size.

3. Unsorted Bin:

- If no match in **tcache** or **fastbin**, **glibc** looks here.
- It might be also possible that memory might have been freed during the process execution. Hence, these free memory needs to go somewhere which is our Unsorted Bin.
- Now, this is **temporary placeholder** for the recently freed

chunks that do not fit in **fastbins** or **tcache (or when tcache/fastbins are full)**

- If a chunk in the unsorted bin is not immediately reused by a subsequent `malloc` call, **glibc** will coalesce it with adjacent free chunks (if any) and then move it to the appropriate small or large bin based on its size.

4. Small Bins

- Hold freed chunks of specific small sizes (larger than fastbins, up to a certain size).
- Each bin holds chunks of a single size (doubly-linked list).
- Used if tcache, fastbin, and unsorted bins do not contain the requested size.

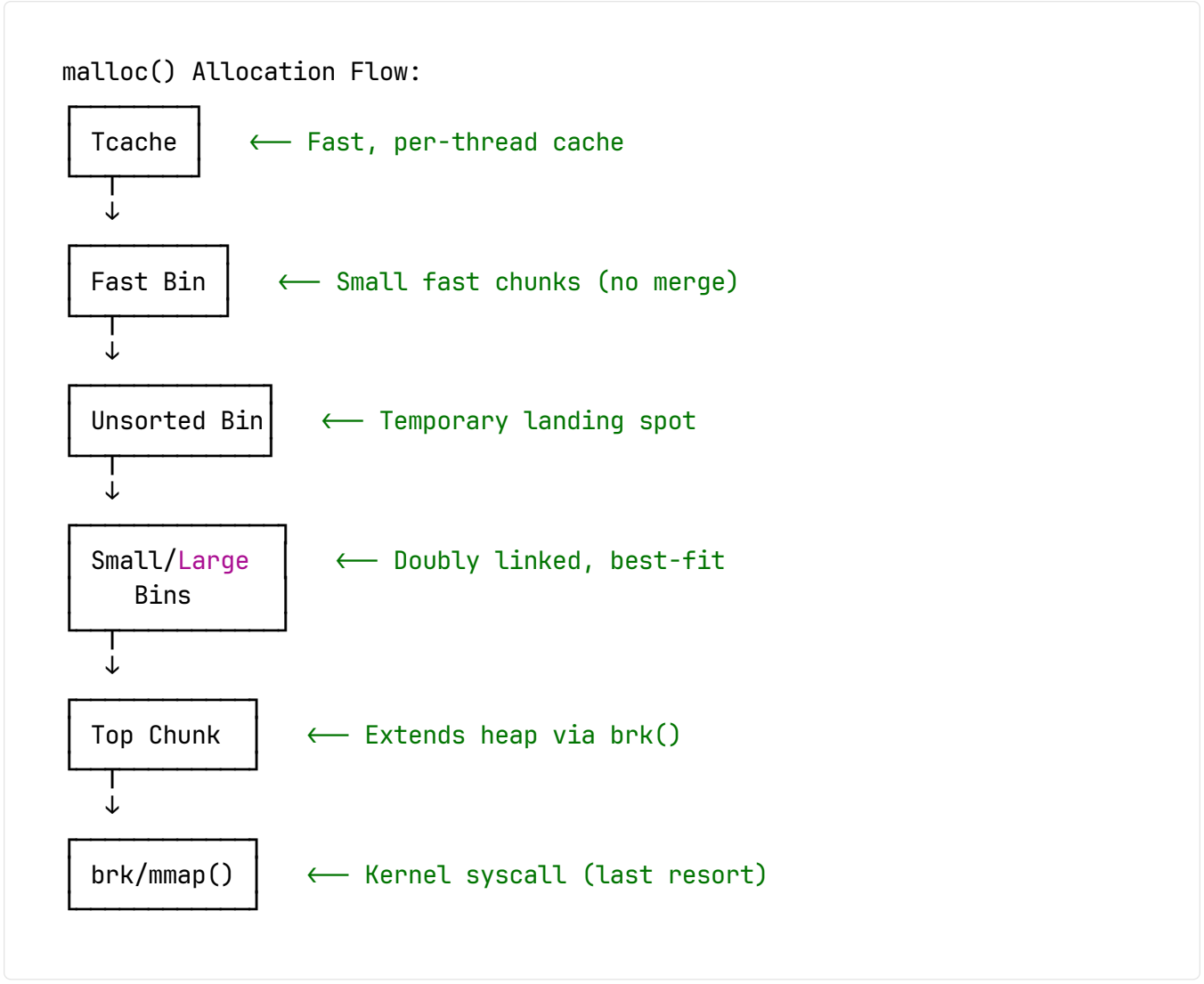
5. Large Bins

- Manage even larger freed chunks.
- Fewer large bins, each covering a range of sizes.
- Like small bins, these are doubly linked lists, maintained for efficient lookup.

Now, lets say from our tcache, fastbins, unsorted bins, small bins, or large bins do not have any suitable free chunks to reuse.

The top chunk represents the remainder of free heap space at the high end of the heap that has never been split into smaller chunks or used yet. It:

For example, if you request **256 bytes** and **the top chunk has 1 KB** free, it splits off **256 bytes** to allocate and retains the remaining **768 bytes as the new top chunk.**



So let's say that even top chunk is not able to provide the requested size.

Now, from here, we might have two scenarios.

Scenario 1: Smaller Allocation

(i) **For small and moderate allocations** (usually below a threshold around **128 KB**, glib will extend the heap using **sbrk() system call.** This increases the heap segment by moving the program break forward, enlarging the top chunk accordingly. Then, the allocation is fulfilled from this expanded top chunk.

- Now, when I say that a process calls **sbrk**() and grow its heap, what eventually happen is the process's extends **virtual memory address** region for the heap.
- Operating systems with virtual memory (like Linux) only allocate physical RAM "frames" when the process actually accesses the corresponding virtual memory pages. This is known as "**demand paging.**"
- When you request more heap via `sbrk()`, the kernel simply records the new end of the heap and sets up virtual memory page mappings. The actual physical frames are assigned by the kernel's memory manager only when the program first attempts to access that memory

Scenario 2: Larger Allocation

For large allocations (above that **threshold**), **glibc** bypasses the **top chunk** and **heap extension** with `sbrk()`. Instead, it calls `mmap()` directly to **allocate** a separate, dedicated memory region to satisfy the request. This keeps large allocations out of the main heap to reduce fragmentation and allows them to be released back to the OS immediately upon free.

The chunk which is allocated by the mmap is marked as IS_MMAPED and lives in its own anonymous mapping. This piece of memory is never split for smaller allocation or places in bins.

The MMAP

The `mmap()` system call's primary job is to **reserve a contiguous range of *virtual* addresses for the calling process.** It then records the requested memory protection (read/write/execute) within a new Virtual Memory Area (VMA) entry in the process's `mm_struct`.

Kernel maintains a data structure for each process called the **`mm_struct`** (memory descriptor).

- The `mm_struct` contains a linked list (or a tree in new kernels) of VMAs (**Virtual Memory Areas**).
- Each VMA describes a contiguous range of virtual addresses with the same access permissions and backing storage (file, anonymous, etc.).
- When **`mmap()`** is called, the kernel creates a new VMA (a `struct vm_area_struct` in the kernel) for the requested range and inserts it into the list/tree in `mm_struct`

Further, there is also one more thing, that happens as part of it, recording the protection bits. Now, this is something that is specified by the user.

Record Protection Bits

- The VMA stores protection bits (e.g., read, write, execute) as specified by the user through the `prot` argument of `mmap()`.
- These bits are saved in the VMA (`vm_page_prot` and related fields).

Important thing to note is that:

→ No actual page tables are updated or created yet!

→ The kernel only updates page tables when the process first accesses a page in the mapped region (on a page fault).

No page table entries are populated yet. The VMA is recorded, but actual page mappings happen lazily upon first access, triggering a page fault.

The first time your code reads or writes a byte in the mapping the CPU triggers a page-fault exception.

The kernel's handler then

- allocates a free physical page (or picks one from the swap cache),
- zero-fills it (for anonymous mappings),
- updates the page table to point to the new frame
- returns to user space.

The instruction is **retried** and now **succeeds.**

This behavior is actually called **lazy, demand-page behavior.** Just because you called malloc, does not mean, you will immediately start writing to the newly allocated address space.

Note: This section is for the people, who wants to dive further.

When the page-fault handler needs one free physical frame. It follows different path.

Before this, I will try to give you an idea how physical memory is internally managed.

In Linux, physical memory is organized into zones (such as **DMA**, **DMA32**, **Normal**), and each zone has its own buddy allocator, which I shall explain later.

These zones exist due to historical **hardware limitations** and **performance considerations**, as different devices may only access certain physical address ranges

```
+----------------------------+  ← 0x00000000
|        ZONE_DMA            |  (< 16 MB)
|                            |  For legacy DMA devices that need
|                            |  low physical addresses.
+----------------------------+
|       ZONE_DMA32           |  (16 MB - 4 GB)
|                            |  For 32-bit-capable DMA devices
|                            |  that can access up to 4 GB.
+----------------------------+
|       ZONE_NORMAL          |  (> 4 GB)
|                            |  Main memory used by the kernel.
|                            |  Directly mapped by the kernel.
+----------------------------+
|      ZONE_MOVABLE          |  (Optional region, within RAM)
|                            |  Used for page migration,
|                            |  hotplug memory, and defragmentation.
+----------------------------+  ← Can extend to terabytes
```

Zone Structure

Each memory zone maintains:

- **Per-CPU page sets**: Quick cache for small allocations (order ≤ 3).

- **Buddy free area**: The main buddy allocator for larger requests.
- **Migration types**: Pages grouped by movability (Unmovable, Movable, Reclaimable)

Now, continuing our path, I shall not trouble you further and tell from where we shall get our page, finally.

1. **Fast path — per-CPU page lists**
   Per-CPU Page Sets are **small, fast caches of free pages** maintained **per CPU core**, specifically for **small allocations** (up to order 3, i.e., ≤ 8 pages = 32 KB on a 4 KB system).

Imagine if every tiny memory allocation had to lock and interact with the **global buddy allocator** — that would:

- **Introduce lock contention**
- Slow things down, especially on multi-core CPUs

Solution: Let each CPU **maintain a small private pool** of free pages for quick reuse

If the per-CPU page set cannot satisfy the request (either because it's empty or the request is too large, e.g., > 32KB), the request is then forwarded to the **buddy allocator**.

Buddy allocator

Buddy memory allocation technique is a memory allocation algorithm that divides memory into partition to satisfy memory request. The system maintains block of memory in the power of 2. **[For example: 4,8,16, 32].**

Internally, the allocator uses a **binary tree** to represent used or unused split, which I will explain in depth.

Let’s say that malloc requested a memory block of size 200 KB.

200 KB must be rounded up to the next available power-of-two block:

- 200 KB → 256 KB (order 6).

Case A: Block of 256 KB is free.

An order 6 (256 KB) block is already free
- Allocate it directly to satisfy the 200 KB request.
- Internal fragmentation: 56 KB inside the 256 KB block remains unused but is contained within the allocation.

Case B — No order 6 block, but a larger one exists (say, one order 7 block of 512 KB)

1. Split the 512 KB block into two 256 KB buddies (order 6).
2. Give one 256 KB block to the requester.
3. Return the other 256 KB block to the order 6 free list.

The power-of-two rule is preserved throughout, so the spare 256 KB and any future free 256 KB block can merge back into 512 KB.

3 . Why we never split into 200 KB + 56 KB

Attempting to carve an exact 200 KB piece from a 256 KB block would break the buddy system:

- Not a power of two — 200 KB and 56 KB cannot be expressed as 2^n × page-size.
- Lost buddy relationship — 200 KB and 56 KB are not “true

buddies", so they can never recombine.

- Permanent fragmentation — The leftover 56 KB could be wasted forever, defeating the allocator's O(log n) split/merge guarantees.

Therefore the allocator always rounds the request up to a **legal block size (here 256 KB)** and keeps any unused space as harmless internal fragmentation inside that block.

Internally buddy allocator will also be maintaining a pool of free pages. Since, request can come at any time, it must ensure that it has enough memory within its pool to serve the request. Think of it, like an e**arly warning system.**

You might have seen a water tank, when the water tank **marking falls below** a level, you know its time to get more water. Linux also follow the same principle. All this happen in the **background**, not during the time the original request has come. There is one little exception to this as well, which I am going to explain at the end of the article.

Let's first start understanding, **watermark level.**

Linux maintains three watermark levels for each zone:

Watermark System

Three Watermark Levels

1. **Min watermark:** Minimum free pages before emergency allocation
2. **Low watermark:** Threshold that triggers **kswapd** (kernel swap daemon)

3. **High watermark**: Level where **kswapd** stops reclaiming memory

The relationships are:

- `watermark[low] = watermark[min] * 5/4`
- `watermark[high] = watermark[min] * 3/2`

Understanding the low watermark breach

1. **Low watermark breach: When free memory drops below the low watermark, kswapd is immediately awakened**

- The **kswapd daemon** is awakened to start reclaiming memory from other process.

`kswapd` tries to **free up memory** by:

1. **Scanning page lists** (LRU — Least Recently Used)
2. Reclaiming pages:

- Pages are taken from the inactive LRU lists.
- **Clean file-backed page** → simply unlock and drop (no write-back).
- **Dirty file-backed page** → schedule write-back, then drop.
- **Anonymous page** → if swap is enabled, write it to the swap area and free the frame; if swap is disabled, the page cannot be reclaimed this way.
  Reclaimed frames go back to the buddy allocator's free lists.

Now, finally, we shall discuss about the scenario, **where even the buddy allocator does not have the available memory.**

The Failure Scenario

Direct reclaim represents a failure of the watermark system:

- kswapd was awakened but couldn’t reclaim memory fast enough
- Memory pressure exceeded kswapd’s ability to keep up
- The system fell below even the minimum watermark levels

Direct Reclaim Triggered

- New allocation request arrives
- Buddy allocator cannot satisfy the request
- The allocating process itself performs direct reclaim
- This happens independently of kswapd’s ongoing work

Direct Reclaim

- **Application blocking**: The requesting application stops and waits
- **System stalls**: Can causenoticeable pauses in system responsiveness
- **Cascading effects**: Multiple applications may hit direct reclaim simultaneously

Now, let’s take things even more destructive, when **even direct reclaim fails to free sufficient memory**. The Linux enters its most cirtical memory management phase.

Before resorting to more drastic measures, the kernel attempts memory compaction:

- **Page migration**: Moves movable pages to create larger contiguous blocks
- **Fragmentation reduction**: Combines smaller free pages into larger allocatable chunks
- **Last optimization effort:** Attempts to satisfy allocation requests through reorganization rather than freeing more memory

Memory compaction may fail when:

- **Insufficient movable pages**: Too many unmovable kernel pages preventing reorganization
- **Severe fragmentation:** Memory is too scattered to create meaningful contiguous blocks
- **Time constraints:** Compaction takes too long and allocation pressure continues

So now, it time to KILLLLLL!!!

OOM Killer Activation

When all memory reclamation efforts fail, the kernel invokes the OOM killer:

- **Process selection**: Identifies processes consuming the most memory. The selection is based on higher memory usage. Newer, memory-hungry processes are preferred targets.
- **Scoring algorithm**: Uses heuristics to determine which processes to terminate
- **Forceful termination**: Kills selected processes to immediately free their memory

Persistent Memory Exhaustion

In extreme cases where even OOM killing cannot help:

- **System instability**: The system may become largely unresponsive
- **Kernel panic**: In some configurations, the kernel may panic rather than continue
- **Hard reboot requirement**: Manual intervention may be necessary to restore functionality

The end, this is all your system can do.

The journey from a simple `malloc` call to the intricate dance of memory allocation across user and kernel space is a complex one, culminating in the robust management of system resources. While the OOM killer serves as a critical safeguard against catastrophic memory exhaustion, it rarely comes into play in a well-managed system. Instead, the sophisticated interplay of glibc's bins, `sbrk()`, `mmap()`, and the kernel's buddy allocator and demand-paging mechanisms work seamlessly in the background. This multi-layered approach ensures that applications receive the memory they need efficiently and reliably, allowing developers to focus on functionality rather than low-level memory intricacies, and making dynamic memory allocation a powerful tool for building scalable and responsive software

Programming Coding Software Development

Follow

Written by Aniket Kumar

39 followers · 61 following

Coder for Fun

How does a malloc work?. Malloc is a standard l... https://cmmon.medium.com/chow-does-a-malloc-wor...

How does a malloc work?. Malloc is a standard l... https://cmmon.medium.com/chow-does-a-malloc-wor...